Pontifícia Universidade Católica de Minas Gerais
Instituto de Ciências Exatas e Informática (ICEI)
Engenharia de Software
Disciplina: Sistemas Operacionais
Professor: Diego Silva Caldeira Rocha

Trabalho Prático 1

Comunicação e Sincronização de Processos e Threads

Integrantes:
Lucas Erick Andrade da Silva
Julia Carvalho
João Paulo de Freitas Fadul
Victhor Guilherme Santos Loiola
Bernardo Demaria Santos

1. Introdução

Neste trabalho nós estudamos como várias threads podem usar o mesmo recurso sem causar erro. Na Parte I, rodamos o exemplo do produtor e consumidor que o professor passou. Na Parte II, fizemos em Java os sete problemas do enunciado, usando threads e semáforos. Cada programa tem um teste que confere se tudo funcionou direito.

Algumas palavras que aparecem no texto:

- **Thread:** uma tarefa do programa que roda ao mesmo tempo que outras.
- **Semáforo:** um contador que controla quem pode continuar. Se o valor é zero, a thread espera.
- **Região crítica:** o pedaço do código que só uma thread pode executar por vez.
- **Deadlock:** quando as threads ficam esperando umas pelas outras e o programa trava.

2. Parte I: Thread em Java

A classe `Buffer` guarda um valor só e uma variável `cheio`. O método `put` só grava quando o buffer está vazio, e o método `get` só lê quando ele está cheio. Os dois são `synchronized`, ou seja, só uma thread entra por vez. Quando não pode continuar, a thread chama `wait()` e fica esperando. Quem termina chama `notifyAll()` para acordar a outra.

O `Produtor` coloca os valores de 0 a 4 e o `Consumidor` retira 5 valores. Como o buffer só guarda um valor, um novo `put` só acontece depois do `get` do valor anterior.

Esta é a saída de uma execução. As linhas `put` e `get` são impressas dentro do buffer, e as linhas `Produtor` e `Consumidor` são impressas fora dele. Por isso a ordem entre elas pode mudar de uma execução para outra, mas os valores sempre saem em ordem (0, 1, 2, 3, 4) e nenhum se perde.

```
put 0
get 0
Produtor #0 put: 0
Consumidor #0 get: 0
put 1
Produtor #0 put: 1
get 1
Consumidor #0 get: 1
put 2
get 2
Consumidor #0 get: 2
Produtor #0 put: 2
put 3
get 3
Consumidor #0 get: 3
Produtor #0 put: 3
put 4
get 4
Consumidor #0 get: 4
Produtor #0 put: 4
```

3. Parte II: Programas com semáforos

Os programas estão na pasta `src/trabalho`. Em cada problema abaixo dizemos quais são as threads, o que elas dividem, qual é a região crítica, quais semáforos usamos (com o valor inicial), como evitamos deadlock e como testamos. Se um programa travar, o teste acusa erro depois de um tempo, em vez de ficar parado para sempre.

3.1 Produtor e consumidor

Threads: 3 produtores e 3 consumidores.
O que dividem: um buffer de 5 posições.
Região crítica: mexer no buffer e nas posições de entrada e saída.
Semáforos: `vazios` (começa em 5, conta as posições livres), `cheios` (começa em 0, conta os itens guardados) e `mutex` (começa em 1, deixa só uma thread mexer no buffer).
Deadlock: a thread sempre espera `vazios` ou `cheios` antes de pegar o `mutex`, e solta o `mutex` antes de avisar a outra. Assim ninguém dorme segurando o `mutex`.
Teste: os 60 itens produzidos são consumidos uma vez só, o buffer nunca passa de 5 itens e termina vazio.

3.2 Barbeiro dorminhoco

Threads: 1 barbeiro e 12 clientes. A barbearia tem 3 cadeiras de espera.
O que dividem: a variável `waiting`, que conta os clientes esperando.
Região crítica: ler e mudar o `waiting`.

Semáforos: `customers` (0, conta os clientes esperando, e o barbeiro dorme nele), `barbers` (0, avisa que o barbeiro está livre), `mutex` (1, protege o `waiting`) e `done` (0, avisa que o corte terminou).
Deadlock: o cliente solta o `mutex` antes de esperar o barbeiro, e o barbeiro solta o `mutex` antes de cortar. Se a barbearia está cheia, o cliente vai embora sem esperar em semáforo nenhum.
Teste: os atendidos mais os que foram embora somam 12, o barbeiro nunca corta dois ao mesmo tempo e a fila nunca passa de 3.

3.3 Leitores e escritores

Threads: 5 leitores e 2 escritores.
O que dividem: o vetor `M`, com 4 números.
Região crítica: escrever no vetor. Só um escritor por vez, e sem leitor junto. Já os leitores podem ler ao mesmo tempo, porque ler não muda nada.
Semáforos: `mutex` (1, protege o contador de leitores), `db` (1, dá acesso exclusivo ao vetor) e `turno` (1, é justo e funciona como uma catraca, para os leitores não deixarem os escritores esperando para sempre).
Deadlock: o primeiro leitor pega o `db` e o último solta. O `mutex` é solto logo depois de mexer no contador, e o `turno` é solto antes de ler ou escrever. Nenhum semáforo fica preso esperando outro.
Teste: o escritor grava o mesmo número nas 4 posições, devagar de propósito. Se um leitor visse números diferentes, ou se um escritor rodasse junto com outra thread, o teste falharia.

3.4 Fumantes

Threads: 1 representante e 3 fumantes (um com tabaco, um com papel e um com fósforos).
O que dividem: a mesa.
Região crítica: colocar e tirar os ingredientes da mesa.
Semáforos: `agente` (1, só deixa o representante repor depois que o fumante terminou) e um semáforo para cada fumante, `fumante[i]` (0, o fumante fica parado até ser chamado).
Deadlock: só um fumante é chamado por vez, e ele sempre devolve o `agente` no fim. Por isso uma thread nunca fica esperando outra que está esperando ela.
Teste: em 15 rodadas, o fumante chamado sempre achou na mesa os dois ingredientes que ele não tem, e foram 15 cigarros no total.

3.5 Filósofos

Threads: 5 filósofos.
O que dividem: 5 palitos.
Região crítica: comer, que usa o palito da direita (p_i) e o da esquerda (p_{i+1}).
Semáforos: `palito[i]` (1 para cada palito) e `sala` (4).
Deadlock: se os 5 pegassem o palito da direita ao mesmo tempo, todos ficariam esperando o da esquerda e o programa travaria. O semáforo `sala` deixa só 4 filósofos tentarem pegar os palitos. Assim sempre sobra um palito livre e pelo menos um filósofo consegue comer.
Teste do deadlock: tiramos a `sala` (valor 5) e fizemos o filósofo esperar um pouco depois

do primeiro palito. O programa travou em 2 de 3 execuções. Com a `sala` em 4, todas terminaram.
Teste: dois vizinhos nunca comem juntos e cada filósofo come 6 vezes.

3.6 Jantar dos selvagens

Threads: 4 selvagens e 1 cozinheiro.
O que dividem: o caldeirão, que tem 5 porções.
Região crítica: se servir e encher o caldeirão.
Semáforos: `mutex` (1, só um selvagem se serve por vez), `vazio` (0, acorda o cozinheiro) e `cheio` (0, o cozinheiro avisa que encheu).
Deadlock: o selvagem que acha o caldeirão vazio acorda o cozinheiro e espera segurando o `mutex`, de propósito, para ninguém se servir de um caldeirão vazio. Não trava porque o cozinheiro não precisa do `mutex` para encher.
Teste: nunca sobra porção negativa, o cozinheiro só enche quando está vazio e, com 24 refeições e 5 porções, acontecem exatamente 4 recargas.

3.7 Três robôs

Threads: Bart, Lisa e Maggie.
O que dividem: a vez de se mexer (`move()`).
Semáforos: `bart` (1, ele começa), `lisa` (0) e `maggie` (0). Só existe uma vez circulando, então só um robô anda por vez e ninguém fica esperando à toa.
Ordem: Bart, Lisa, Maggie, Lisa, Bart, e assim por diante. A Lisa anda duas vezes por ciclo: no primeiro movimento ela libera a Maggie e no segundo ela libera o Bart.
Deadlock: depois de se mexer, cada robô passa a vez para o próximo, então ninguém espera em círculo.
Teste: em 5 ciclos, os 20 movimentos seguem exatamente a ordem Bart, Lisa, Maggie, Lisa.

4. Testes

Rodamos cada programa 20 vezes seguidas, porque erros de sincronização às vezes só aparecem em algumas execuções. Todos passaram nas 20.

```
Produtor/Consumidor          20/20 execucoes OK
Barbeiro Dorminhoco          20/20 execucoes OK
Leitores e Escritores        20/20 execucoes OK
Fumantes                     20/20 execucoes OK
Filosofos                    20/20 execucoes OK
Selvagens                    20/20 execucoes OK
Tres Robos                   20/20 execucoes OK
TODOS OS TESTES PASSARAM
```

A saída completa de cada programa está na pasta `saidas`.

5. Como compilar e executar

```
javac -d out $(find src -name "*.java")
java -cp out ProdutorConsumidor.ProdCons     # Parte I
java -cp out trabalho.Main 1                 # problemas de 1 a 7
java -cp out trabalho.Main todos             # resumo de todos
java -cp out trabalho.TestesTodos            # 20 execucoes de cada
```

Números dos problemas: 1 produtor e consumidor, 2 barbeiro, 3 leitores e escritores, 4 fumantes, 5 filósofos, 6 selvagens, 7 três robôs.